TECHNICAL ARCHITECTURE

StockPulse Web App

How the dashboard, API, market providers, and Kronos forecast path fit together for local and LAN hosting - including which model is used and how predictions are produced from OHLCV through to the chart.

Date: 24 August 2026

Product: StockPulse (Kronos monorepo)

Local UI: http://localhost:5173

LAN UI: http://192.168.86.197:5173

> Not investment advice. Forecasts are research outputs. Broker acceptance does not guarantee execution. Prefer paper mode until you understand the risk controls.

1. Purpose

StockPulse is a paper-first trading workstation in the Kronos monorepo. The UI shows quotes, charts, news, sentiment, predicted movers, and open positions. Orders are always manual. Forecasts never place trades.

This document describes the web application architecture only. The research multi-model package under forecasting/ and the portfolio engine under kronos_backtest/ are adjacent systems, noted where they touch StockPulse.

2. Runtime topology

Process	Bind	Role
Vite preview (frontend)	0.0.0.0:5173	Static UI + reverse proxy for /api
Uvicorn (backend)	127.0.0.1:8000	FastAPI. Loopback only.
Research forecast API	optional :8001	forecasting.api.serve (not StockPulse)

LAN clients open the frontend URL. They never talk to :8000 directly. The Vite preview server proxies /api/v1/* to the loopback API. That keeps broker keys and model weights off the public interface.

Request path

```
Browser (localhost or LAN)
  -> http://<host>:5173
       -> React dashboard (static build)
       -> /api/v1/*  --proxy-->  FastAPI 127.0.0.1:8000
            -> Alpaca (bars, quotes, trading)
            -> Finnhub (news, public sentiment)
            -> KronosService (forecast / movers)
```

3. Repository layers

Layer	Path	Responsibility
UI	frontend/	React + TypeScript + Vite. Chart, ticket, movers, portfolio.
API	backend/app/	Routes, schemas, DI, rate limits, risk checks.
Providers	backend/app/services/providers.py	Alpaca + Finnhub clients.
Forecast	backend/app/services/kronos.py	Bars windowing, Kronos/baseline predict, annotate.
Research	backend/app/services/research.py	Costs, regime, walk-forward, edge score.
Model	model/	Kronos tokenizer + predictor (HF weights).
Publish	scripts/publish-kronos-lan.ps1	Build UI, start API + preview, print LAN URL.

4. Frontend composition

- App.tsx owns symbol, short/long horizon, forecast bar count, paper/live mode.
- MarketChart draws historical candles and the forecast close path.
- DecisionPanel summarizes path turns, net change, regime, and news context.
- MoversPanel polls a background scan for top predicted gainers/losers.
- PortfolioPanel lists open positions; clicking a symbol reloads the market pane.
- api.ts maps snake_case API payloads to camelCase UI types; coalesces in-flight forecasts.

Partial-data banners appear when overview succeeds but chart or forecast fails (including rate limits). Forecast failures now surface the API error text when present.

5. Which model StockPulse uses

The chart toggle is Kronos (single Kronos model, engine=kronos) or Forecast (weighted ensemble from forecasting/, engine=ensemble). Chronos, TimesFM, and Lag-Llama participate only when installed and enabled; missing adapters are skipped. The optional research API on port 8001 is separate and not used by the dashboard.

Default checkpoint

Setting	Default	Meaning
KRONOS_MODEL_ID	NeoQuasar/Kronos-small	Candlestick foundation model (~24.7M params).
KRONOS_TOKENIZER_ID	NeoQuasar/Kronos-Tokenizer-base	Must match the model family.
KRONOS_DEVICE	auto	cuda / MPS / cpu chosen at load time.
KRONOS_MAX_CONTEXT	512	Hard cap for Kronos-small context window.

Weights download from Hugging Face on the first successful forecast that needs them. Tokenizer and model are loaded once, kept in memory, and protected by an inference lock. Override IDs in backend/.env if you switch to Kronos-mini (longer context) or another compatible checkpoint - keep tokenizer and model sizes matched.

What Kronos is

- Candlestick-native: consumes OHLCV (plus a synthetic amount = volume x mean price).
- Tokenizer encodes continuous bars into discrete tokens (binary spherical quantization).
- Autoregressive decoder predicts future tokens, then decodes them back to OHLCV.
- StockPulse calls KronosPredictor.predict with sample_count=1 (single path, not a Monte Carlo cloud).
- Sampling defaults inside the predictor: temperature T=1.0, top_k=0, top_p=0.9.

When Kronos cannot load

If PyTorch fails (for example Windows Application Control blocking torch DLLs), KronosService switches to baseline-drift: average of the last up to 20 close returns, clamped to +/-2% per bar, applied repeatedly for the horizon. Response model.id is baseline-drift and model.fallback is true. Real Kronos resumes once Torch loads.

6. How a prediction is produced

End-to-end data flow

```
UI: Short|Long + bar count (1/10/20/30/60)
  POST /api/v1/forecast { symbol, preset, horizon, engine }

1. Resolve preset
     short -> 5Min bars, context default 256
     long  -> 1Day bars, context default 256
     horizon from request or preset default (12 / 20)
     context capped at min(requested, kronos_max_context, 512)

2. Fetch history (Alpaca)
     Need at least 32 bars; keep the last `context` bars
     Columns: open, high, low, close, volume
     amount synthesized for the model

3. Build time axes
     x_timestamp = historical bar times (UTC)
     y_timestamp = future session times (ET calendar):
       daily: next weekdays
       intraday: step 5/15/... min, roll to next 09:30 after 16:00

4. Infer path
     try KronosPredictor.predict(df, x_ts, y_ts, pred_len=horizon)
     else baseline-drift on close returns

5. Annotate for the UI
     gross change = last_pred_close / last_hist_close - 1
     net change   = gross - round_trip_cost_bps/10000
     regime from recent vol/trend
     optional walk-forward folds (next-open fill)
     path_segments for DecisionPanel

6. Cache key (symbol, preset, timeframe, context, horizon) TTL 300s
```

Presets vs UI bar picker

Control	Effect
Short / Long toggle	Chooses bar timeframe (5Min vs 1Day) and default horizon.
Bar count buttons	Overrides horizon only (how many future bars to draw).
Context	How many past bars Kronos may see; not user-editable in the UI.

Inputs the model actually sees

- A float DataFrame of open/high/low/close/volume/amount for the context window.
- Aligned historical timestamps and synthesized future timestamps (no future OHLCV).

- pred_len = horizon; StockPulse does not pass true future prices into the model.

Outputs the UI plots

- forecast[]: timestamped predicted OHLCV rows (chart uses close).
- trend.direction / forecast_change / net_forecast_change after cost haircut.
- model.id, device, context, horizon, and fallback flag.
- evaluation: walk-forward hit rate / IC-style stats when folds are enabled.
- Investors sentiment badge follows forecast direction; dimmed if edge is not reliable.

Movers scan (same model path)

POST /forecast/movers runs the same KronosService.forecast path over a universe (Alpaca actives/movers, capped), with evaluate=False for speed, then ranks by absolute net forecast change. Separate rate-limit bucket from per-ticker chart forecasts.

7. Backend services

Dependency injection

On startup, main.py builds Settings from backend/.env, constructs AlpacaService, FinnhubService, and KronosService, and stores them on app.state. Routes receive a Services object via FastAPI Depends.

8. External providers

Provider	Used for	Failure mode
Alpaca	Bars, quotes, clock, paper/live trading, news	502/503; UI partial data
Finnhub	Symbol search assist, news, public sentiment	Optional; overview still returns
Hugging Face	Kronos-small + tokenizer weights	Falls back to baseline-drift

9. Security and hosting

- API binds loopback only; secrets stay on the host.
- Optional API_KEY requires X-API-Key on all routes except /health.
- Live trading requires ALLOW_LIVE_TRADING plus typed LIVE confirmation.
- CORS is restricted; LAN access is via the frontend origin on :5173.
- Rate limits: forecasts, movers scans, and orders use separate buckets.
- Publish with scripts/publish-kronos-lan.ps1 after app changes (checks then build).

10. Related packages (out of StockPulse hot path)

Package	Role vs webapp
forecasting/	Model-agnostic research layer (registry, ensemble, eval). Separate :8001 API.
kronos_backtest/	Look-ahead-safe portfolio backtester. Not called by the dashboard.
webui/	Legacy Flask demo. Not required for StockPulse.

11. Key endpoints

Method	Path	UI use
GET	/health	Liveness / publish check
GET	/stocks/{sym}/overview	Quote, news, public sentiment
GET	/stocks/{sym}/bars	Chart candles
POST	/forecast	Kronos (kronos) or Forecast (ensemble)
POST	/forecast/movers	Start movers scan
GET	/forecast/movers/status	Poll gainers/losers
GET	/positions	Open positions table
POST	/orders/equity	Manual equity order (after review)